

 PERSISTENCE ARCHITECTURE

TicketWave — Data Model & Specifications

An architectural overview of relational schemas, domain entities, status state machines, concurrency controls, and Liquibase migrations.

SECTION 1

Domain Entities & Relationships

Single source of truth enforced via Liquibase migrations and validated at startup.

| User & Profile Domain



User & Identity (`users`)

Core identity entity with BCrypt password hashing and assigned `UserRole` .

Carries a **nullable partner FK**: null indicates a standalone system operator, supporting clean multi-tenancy scoping via `TenantScope` .



Passengers & Preferences

Passenger (`passengers`): Saved traveller profiles owned by a User, reused across multiple bookings with mandatory ownership filters.

UserPreferences (`user_preferences`): Created lazily on first access per user rather than bloating initial registration flows.

| Catalog & Seat Inventory



Routes & Schedules

Route (`routes`): Defines trip parameters (`FLIGHT` , `BUS` , `TRAIN` , `EVENT`) with origins/destinations or venue details.

Schedule (`schedules`): Specific dated instance containing departure, arrival, base fare, currency, vehicle, and driver allocations.



Seat Management (`seats`)

Represents individual bookable units on a schedule, backed by `SeatStatus` and a `BigDecimal(6,3)` price modifier.

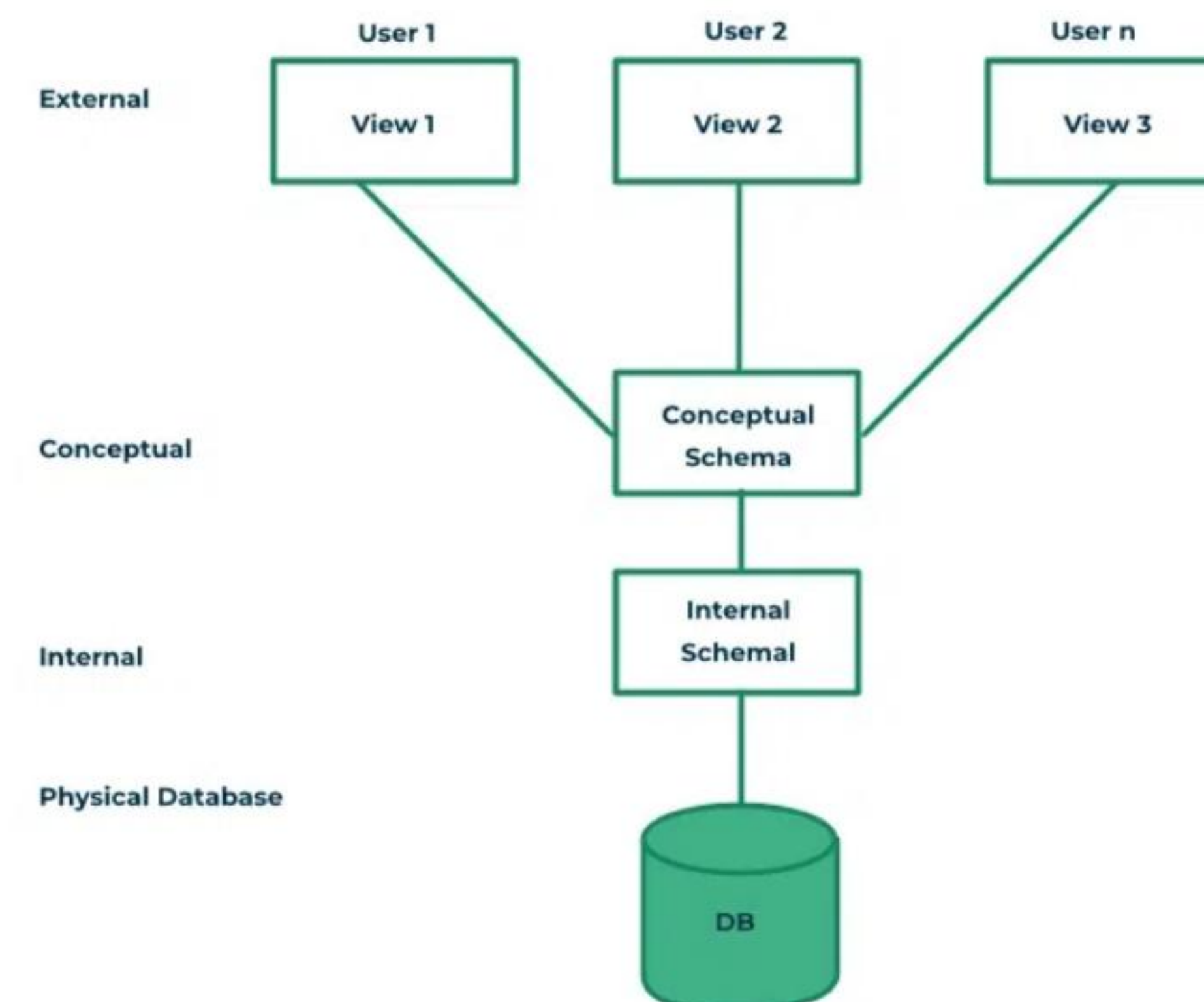
Contains `heldUntil` timestamps and `heldBy` references to allow idempotent re-affirmation while locking out others.

| Booking & Financial Transaction Core

 **Booking & Items:** Carries a unique 10-char PNR, total amount in `BigDecimal(12,2)`, optional idempotency key, and `@Version` field. Item fares are frozen at booking.

 **Payment (`payments`):** References unique transactions. Raw card numbers are **never persisted**—evaluated strictly in-memory for zero PCI leakage.

 **Refund Policy:** Tracks payment refunds with agent override support (`overrideDelta` and `overrideReason`) for fee waivers.



| Ledger, Audit & Partner Integration



Ledger Entries

Append-only financial record (`ledger_entries`). Refunds are stored as negative amounts to guarantee deterministic balances.



Audit Log System

Append-only audit trail without Foreign Keys so log records survive entity deletions. Indexed specifically for fast administrative search.



Partner Webhooks

OAuth2 API credentials hashed with BCrypt. Webhook signing keys stored in **plaintext** to compute outbound HMAC-SHA256 signatures.

SECTION 2

Enums & Persistence Mechanics

Explicit AttributeConverters guarantee readable string columns immune to ordinal shifts.

Persisted Domain Enumerations

Enumeration	Mapped Database Values	Business Purpose
UserRoLe	CUSTOMER, OPERATOR, SUPPORT, ADMIN	Role-based Access Control (RBAC) boundaries
SeatStatus	AVAILABLE, HELD, BOOKED, BLOCKED, RESERVED_OPERATOR	Distinguishes customer holds from operator blocks
BookingStatus	INITIATED, PAYMENT_PROCESSING, CONFIRMED, FAILED, CANCELLED	Tracks lifecycle of ticket reservations
PaymentStatus	PENDING, PENDING_3DS, SUCCEEDED, FAILED, REFUNDED	Enforces payment processor state flow
LedgerEntryType	PAYMENT, REFUND, ADJUSTMENT	Financial audit line item classification

SECTION 3

Concurrency & Data Integrity

Four distinct protection mechanisms securing seat inventory and financial consistency under high load.

Four Pillars of Concurrency Control



Pessimistic Locks

`findByIdForUpdate` locks seat rows. Locks are acquired in **ascending ID order** to eliminate deadlocks across racing checkouts.



Optimistic Locking

`Booking.version` guards against race conditions on refunds (mapped to `409 CONCURRENT_UPDATE` with 3x retry proxy).



Unique Constraints

Database constraints act as ultimate truth for PNR uniqueness, payment references, and booking idempotency keys.

Liquibase Migration Pipeline

33 Chronological Changelogs

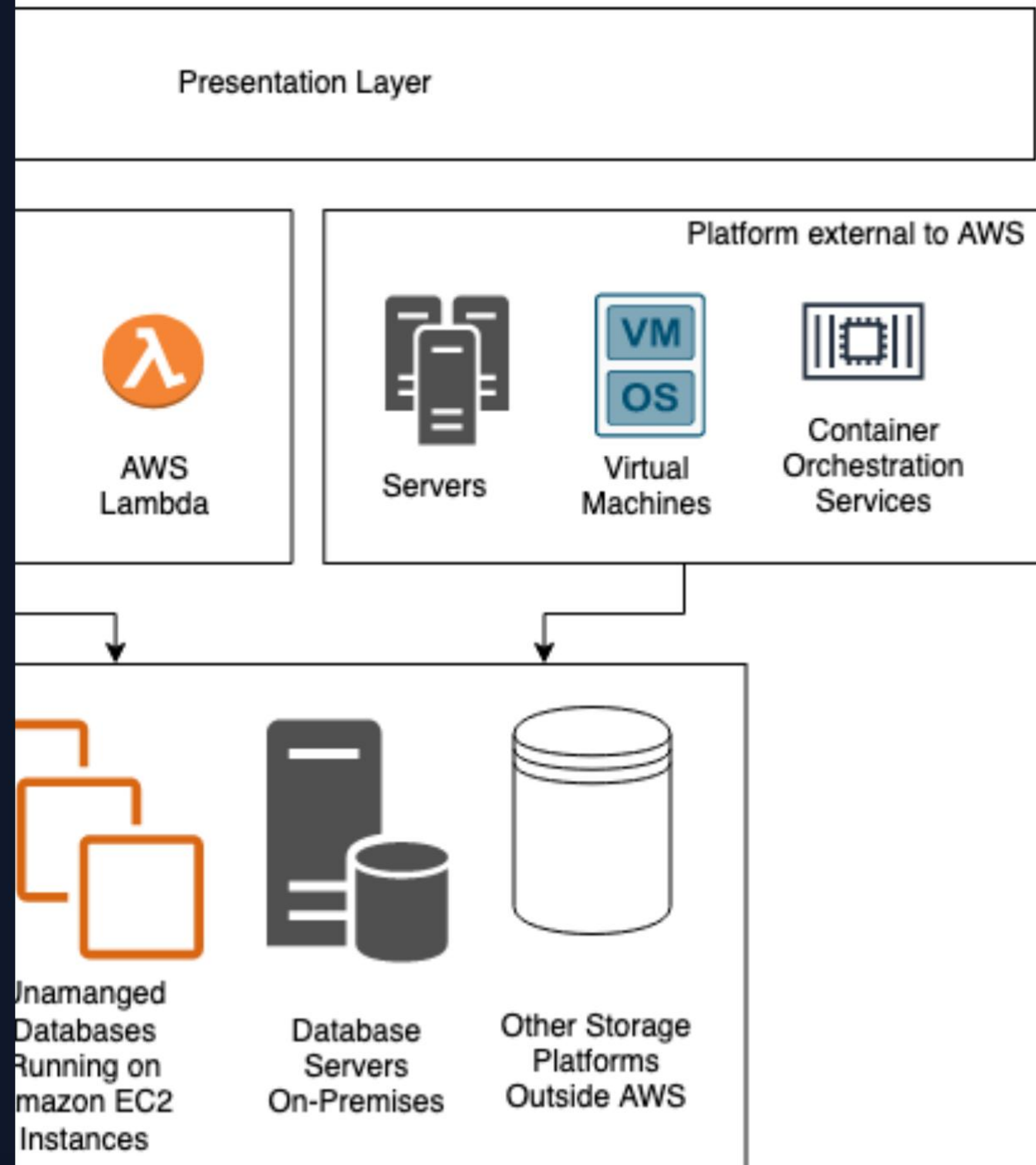
Schema changes are versioned across 33 single-responsibility changelogs. Modification of shipped changesets is strictly prohibited.

Primary Datasource Binding

Migrations are explicitly pinned via `spring.liquibase.url` to the primary database, ensuring read replicas are never targeted during deployment.

PostgreSQL Verification

Integration tests run against real PostgreSQL instances via Testcontainers to validate dialect-specific features and locking behavior.



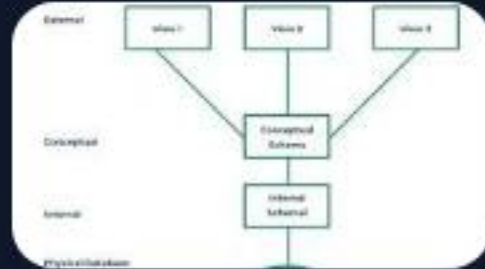
✔ TICKETWAVE DATA MODEL VERIFIED

Questions & Discussion

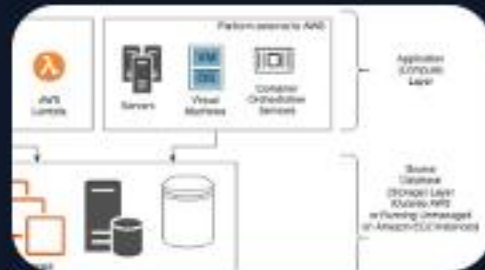
Schema validated via startup check: ddl-auto: validate

`db/changelog/ | backend/src/main/resources`

Image Sources



Source: www.geeksforgeeks.org



Source: aws.amazon.com